

# The Coding Interview Patterns Playbook

Taras Goriachko

# **Section 1 — Course Introduction**

# Requirements

- ▶ Basic knowledge of any programming language - ideally Kotlin
- ▶ Optional: Understanding of the Gradle build system\*
- ▶ **Preinstalled on computer:**
  - ▶ IntelliJ IDEA 2024.2.4 (Community Edition)\*\*
  - ▶ Java 17\*\*\*

---

\* <http://udemy.com/course/mastering-gradle/>

\* <https://www.jetbrains.com/idea/download/?section=mac>

\*\* <https://adoptium.net/en-GB/temurin/releases/?version=17&os=mac&arch=aarch64&package=jre>

# Welcome to the Course

- ▶ Why "Patterns" instead of "Problems"?
- ▶ Moving from memorization to intuition
- ▶ The 80/20 Rule: 15 patterns solve 80% of interviews

*"Don't practice until you get it right. Practice until you can't get it wrong."*

# Welcome to the Course

- ▶ Why "Patterns" instead of "Problems"? 🧠
- ▶ Moving from memorization to intuition
- ▶ The 80/20 Rule: 15 patterns solve 80% of interviews

*"Don't practice until you get it right. Practice until you can't get it wrong."*

# Welcome to the Course

- ▶ Why "Patterns" instead of "Problems"? 🧠
- ▶ Moving from memorization to intuition 💡
- ▶ The 80/20 Rule: 15 patterns solve 80% of interviews

*"Don't practice until you get it right. Practice until you can't get it wrong."*

# Welcome to the Course

- ▶ Why "Patterns" instead of "Problems"? 🧠
- ▶ Moving from memorization to intuition 💡
- ▶ The 80/20 Rule: 15 patterns solve 80% of interviews 📈

*"Don't practice until you get it right. Practice until you can't get it wrong."*

# How to Use This Course Effectively

- ▶ **Course Approach:** We will start by showing the brute-force solution, and then transition to the optimal solution.
- ▶ **In an Interview (Ideal Case):**
  1. **Identify:** Recognize pattern triggers in the problem text
  2. **Trace:** Dry-run the logic on a small input
  3. **Implement:** Code the solution without looking at hints
- ▶ **If Stuck:** Start with a brute-force solution, optimize what you can, and carefully listen to your interviewer.



# How to Use This Course Effectively

- ▶ **Course Approach:** We will start by showing the brute-force solution, and then transition to the optimal solution.
- ▶ **In an Interview (Ideal Case):**
  1. **Identify:** Recognize pattern triggers in the problem text 🔍
  2. **Trace:** Dry-run the logic on a small input
  3. **Implement:** Code the solution without looking at hints
- ▶ **If Stuck:** Start with a brute-force solution, optimize what you can, and carefully listen to your interviewer.

# How to Use This Course Effectively

- ▶ **Course Approach:** We will start by showing the brute-force solution, and then transition to the optimal solution.
- ▶ **In an Interview (Ideal Case):**
  1. **Identify:** Recognize pattern triggers in the problem text 🔍
  2. **Trace:** Dry-run the logic on a small input 🖋️
  3. **Implement:** Code the solution without looking at hints
- ▶ **If Stuck:** Start with a brute-force solution, optimize what you can, and carefully listen to your interviewer.

# How to Use This Course Effectively

- ▶ **Course Approach:** We will start by showing the brute-force solution, and then transition to the optimal solution.
- ▶ **In an Interview (Ideal Case):**
  1. **Identify:** Recognize pattern triggers in the problem text 🔍
  2. **Trace:** Dry-run the logic on a small input 🖋️
  3. **Implement:** Code the solution without looking at hints 🖥️
- ▶ **If Stuck:** Start with a brute-force solution, optimize what you can, and carefully listen to your interviewer.

# Interview Mindset: Thinking in Patterns

- ▶ Stop looking at the "Story" (e.g., "Alice has 3 apples...")
- ▶ Start looking at the **Data Flow**

| Problem Trait                     | Pattern Strategy |
|-----------------------------------|------------------|
| Finding a sub-segment in a list   | Sliding Window   |
| Searching a sorted range          | Binary Search    |
| Shortest path in unweighted graph | BFS              |

# Common Interview Mistakes

- ▶ **Mistake 1:** Starting to code before the logic is 100% clear
- ▶ **Mistake 2:** Forgetting to state Big O complexity
- ▶ **Mistake 3:** Not testing for  $n = 0$  or  $n = 1$

**Correction:** Think out loud. Your interviewer is your collaborator, not your judge.

# Common Interview Mistakes

- ▶ **Mistake 1:** Starting to code before the logic is 100% clear ❌
- ▶ **Mistake 2:** Forgetting to state Big O complexity
- ▶ **Mistake 3:** Not testing for  $n = 0$  or  $n = 1$

**Correction:** Think out loud. Your interviewer is your collaborator, not your judge.

# Common Interview Mistakes

- ▶ **Mistake 1:** Starting to code before the logic is 100% clear ✖
- ▶ **Mistake 2:** Forgetting to state Big O complexity ✖
- ▶ **Mistake 3:** Not testing for  $n = 0$  or  $n = 1$

**Correction:** Think out loud. Your interviewer is your collaborator, not your judge.

# Common Interview Mistakes

- ▶ **Mistake 1:** Starting to code before the logic is 100% clear ✖
- ▶ **Mistake 2:** Forgetting to state Big O complexity ✖
- ▶ **Mistake 3:** Not testing for  $n = 0$  or  $n = 1$  ✖

**Correction:** Think out loud. Your interviewer is your collaborator, not your judge.



# Course Structure

We will discuss the most common patterns and strategies.

## **Section 2 — Fundamentals & Big O**

# The Absolute Essentials

Before mastering coding patterns, you must master the fundamental building blocks.

- ▶ **Why?** Interviewers evaluate your ability to select the right tool for the job.
- ▶ **Key Areas:**
  - ▶ Analyzing efficiency (Big O Complexity).
  - ▶ Choosing the right data structures.
  - ▶ Writing idiomatic, clean code under pressure.

# Big O Notation Demystified

How does the runtime or space requirement grow as input size  $N$  increases?

- ▶ **Time Complexity:** Count the number of dominant operations.
  - ▶  $O(1)$  - Constant time (direct access).
  - ▶  $O(\log N)$  - Logarithmic (halving the search space).
  - ▶  $O(N)$  - Linear (single pass).
  - ▶  $O(N \log N)$  - Linearithmic (sorting).
  - ▶  $O(N^2)$  - Quadratic (nested loops).
- ▶ **Space Complexity:** Measure auxiliary memory used (call stack, maps, arrays) excluding input size itself.

# Data Structures Crash Course

| Data Structure       | Strengths                            | Common Use Case                |
|----------------------|--------------------------------------|--------------------------------|
| <b>Array / List</b>  | Fast index access ( $O(1)$ )         | Ordered traversal, sorting     |
| <b>HashMap / Map</b> | Key lookup ( $O(1)$ average)         | Frequency counting, cache      |
| <b>HashSet / Set</b> | Duplicate removal, search ( $O(1)$ ) | Visited trackers, unique items |
| <b>Queue / Deque</b> | FIFO ordering ( $O(1)$ push/pop)     | BFS, level order traversal     |

# Writing Idiomatic Code under Pressure

Write production-ready, readable code rather than terse competitive code.

- ▶ **Kotlin Idioms:**

- ▶ Use 'maxOf(a, b)' and 'minOf(a, b)' instead of custom ifs.
- ▶ Prefer 'when' expressions for clean decision branches.
- ▶ Avoid force-unwrapping nullables ('!i') unless absolutely safe.

- ▶ **Self-Documentation:** Use descriptive variable names ('left', 'right', 'charCount') instead of obscure ones ('l', 'r', 'c').

## **Section 3 — Two Pointers Pattern**

# What is the Two Pointers Pattern?

- ▶ **The Core Idea:** Use two indices (pointers) to iterate through a data structure simultaneously.
- ▶ **Why it works:** It reduces  $O(n^2)$  nested loops to a single  $O(n)$  pass.
- ▶ **Common Triggers:**
  - ▶ The array is **Sorted**.
  - ▶ You need to find a pair, a triplet, or a subarray.
  - ▶ You need to "filter" or "rearrange" an array in-place.



# Pattern Mechanics: Opposite Ends

When searching for a target in a **Sorted** array:

1. **Left Pointer:** Starts at index 0.
2. **Right Pointer:** Starts at index  $n - 1$ .
3. **Decision:**
  - ▶ If  $sum > target$ : Move Right pointer left ( $\leftarrow$ ) to decrease sum.
  - ▶ If  $sum < target$ : Move Left pointer right ( $\rightarrow$ ) to increase sum.

## **Problem 1: Two Sum**

# Problem 1: Two-Pointer Approach

**Problem:** Find indices of two numbers in a **sorted** array that sum to a target.

## The Algorithm

1. Initialize  $i = 0, j = n - 1$
2. Calculate  
 $sum = nums[i] + nums[j]$
3. If  $sum == target$ , return indices.
4. If  $sum < target$ , move  $i \rightarrow$
5. If  $sum > target$ , move  $j \leftarrow$

## Kotlin Implementation

```
fun twoSum(nums: IntArray, target: Int):  
    IntArray {  
    var i = 0  
    var j = nums.size - 1  
  
    while (i < j) {  
        val sum = nums[i] + nums[j]  
        when {  
            sum == target -> return  
                intArrayOf(i, j)  
            sum < target -> i++  
            else -> j--  
        }  
    }  
    return intArrayOf()}
```

## Complexity

- ▶ **Time:**  $O(N)$
- ▶ **Space:**  $O(1)$

## Two Sum: Visual Trace

**Target = 35**

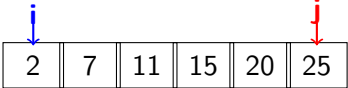
1. 

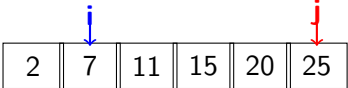
|   |   |    |    |    |    |
|---|---|----|----|----|----|
| 2 | 7 | 11 | 15 | 20 | 25 |
|---|---|----|----|----|----|

 $2 + 25 = 27$  (Too Low,  $i++$ )

## Two Sum: Visual Trace

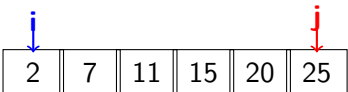
**Target = 35**

1.   $2 + 25 = 27$  (Too Low,  $i++$ )

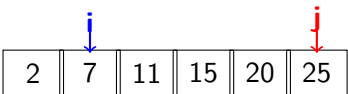
2.   $7 + 25 = 32$  (Too Low,  $i++$ )

## Two Sum: Visual Trace

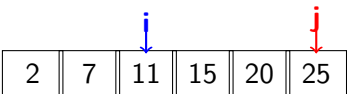
**Target = 35**

1.   $2 + 25 = 27$  (Too Low,  $i++$ )

The diagram shows an array of six numbers: 2, 7, 11, 15, 20, 25. A blue arrow labeled 'i' points to the first element (2), and a red arrow labeled 'j' points to the last element (25).

2.   $7 + 25 = 32$  (Too Low,  $i++$ )

The diagram shows the same array. The blue arrow 'i' has moved to the second element (7), while the red arrow 'j' remains at the last element (25).

3.   $11 + 25 = 36$  (Too High,  $j--$ )

The diagram shows the same array. The blue arrow 'i' has moved to the third element (11), and the red arrow 'j' has moved to the fourth element (20).

## Two Sum: Visual Trace

**Target = 35**

1. 

|   |   |    |    |    |    |
|---|---|----|----|----|----|
| 2 | 7 | 11 | 15 | 20 | 25 |
|---|---|----|----|----|----|

 $2 + 25 = 27$  (Too Low,  $i++$ )

2. 

|   |   |    |    |    |    |
|---|---|----|----|----|----|
| 2 | 7 | 11 | 15 | 20 | 25 |
|---|---|----|----|----|----|

 $7 + 25 = 32$  (Too Low,  $i++$ )

3. 

|   |   |    |    |    |    |
|---|---|----|----|----|----|
| 2 | 7 | 11 | 15 | 20 | 25 |
|---|---|----|----|----|----|

 $11 + 25 = 36$  (Too High,  $j--$ )

4. 

|   |   |    |    |    |    |
|---|---|----|----|----|----|
| 2 | 7 | 11 | 15 | 20 | 25 |
|---|---|----|----|----|----|

 **$15 + 20 = 35!$**

## Problem 2: 3Sum



## **Problem 3: Container With Most Water**

# Container With Most Water

**Goal:** Find two lines that together with the x-axis forms a container, such that the container contains the most water.

## The Algorithm

1.  $i = 0, j = n - 1$ .
2.  $Area = \max(h[i], h[j]) \times (j - i)$ .
3. Update *maxArea* if current is larger.
4. **Crucial:** Move the pointer pointing to the **shorter** line.

## Kotlin Implementation

```
fun maxArea(height: IntArray): Int {  
    var max = 0  
    var i = 0  
    var j = height.size - 1  
  
    while (i < j) {  
        val h = minOf(height[i], height[j])  
        val w = j - i  
        max = maxOf(max, h * w)  
  
        if (height[i] < height[j]) i++  
        else j--  
    }  
    return max  
}
```

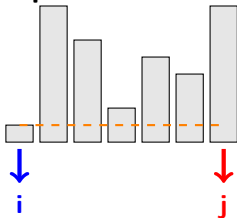
## Complexity

- ▶ **Time:**  $O(N)$
- ▶ **Space:**  $O(1)$

# Container With Most Water: Trace

Heights: [1, 8, 6, 2, 5, 4, 8]    Max Area: 40

## Step 1: Initial State



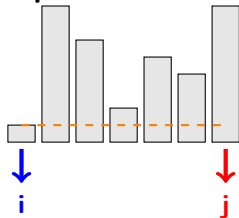
$W = 6, H = 1 \rightarrow \text{Area} = 6$

$h[i] < h[j] \Rightarrow i++$

# Container With Most Water: Trace

Heights: [1, 8, 6, 2, 5, 4, 8]    Max Area: 40

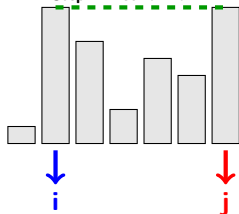
## Step 1: Initial State



$W = 6, H = 1 \rightarrow \text{Area} = 6$

$h[i] < h[j] \Rightarrow i++$

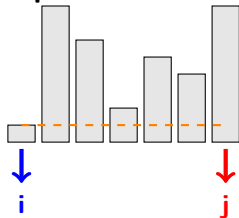
## Step 2: Found Max



# Container With Most Water: Trace

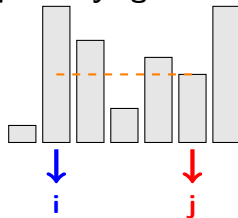
Heights: [1, 8, 6, 2, 5, 4, 8]    Max Area: 40

Step 1: Initial State



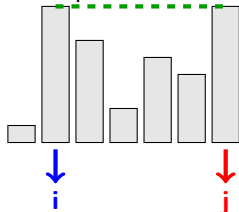
$W = 6, H = 1 \rightarrow \text{Area} = 6$   
 $h[i] < h[j] \Rightarrow i++$

Step 3: Trying for more...



$W = 4, H = 4 \rightarrow \text{Area} = 16$   
Max remains 40

Step 2: Found Max

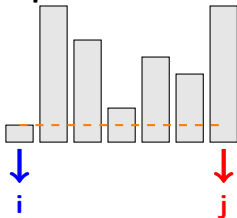


$W = 5, H = 8 \rightarrow \text{Area} = 40$

# Container With Most Water: Trace

Heights: [1, 8, 6, 2, 5, 4, 8]    Max Area: 40

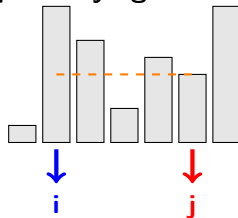
## Step 1: Initial State



$$W = 6, H = 1 \rightarrow \text{Area} = 6$$

$h[i] < h[j] \Rightarrow i++$

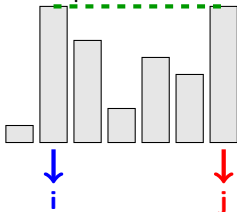
## Step 3: Trying for more...



$$W = 4, H = 4 \rightarrow \text{Area} = 16$$

Max remains 40

## Step 2: Found Max



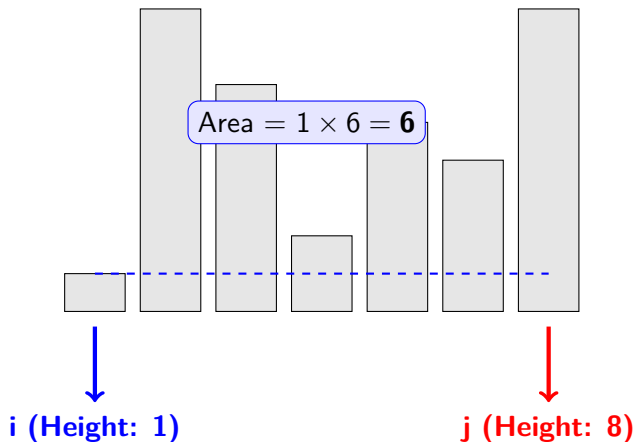
$$W = 5, H = 8 \rightarrow \text{Area} = 40$$

## Intuition

Moving the taller pointer can only decrease area (width shrinks and height is capped by the shorter side). We **must** move the shorter pointer to find a taller container.

## Container With Most Water: Iteration 1

Heights: [1, 8, 6, 2, 5, 4, 8]



Logic

$h[i] < h[j]$  ( $1 < 8$ ). To find a larger area, we move the **shorter**

## **Problem 4: Remove Duplicates from Sorted Array**



## Two Pointers – Key Takeaways

- ▶  **Space Efficiency:** Usually allows for  $O(1)$  extra space.
- ▶  **In-place operations:** Perfect for "Move Zeroes" or "Remove Duplicates."
- ▶  **Pointer Variants:**
  - ▶ **Opposite Ends:** (Left, Right) → Searching/Sorting.
  - ▶ **Same Direction:** (Slow, Fast) → Removing elements.

## **Section 4 — Sliding Window**

# What is the Sliding Window Pattern?

- ▶ **The Core Idea:** Maintain a contiguous window over a linear data structure (array/string) that dynamically expands or contracts.
- ▶ **Why it works:** Converts nested  $O(n^2)$  loops searching subarrays into a single  $O(n)$  pass by reusing computed information.
- ▶ **Common Triggers:**
  - ▶ Subarray or substring problems.
  - ▶ Finding shortest, longest, or target length matching constraint.
  - ▶ Tracking unique elements/frequencies in a range.

# Dynamic vs. Fixed Window

- ▶ **Fixed Window:** Size  $K$  is constant. Move window by adding next element and removing the leftmost.
- ▶ **Dynamic Window:** Size changes.
  - ▶ Expand the window by moving 'right' pointer.
  - ▶ If constraint is violated, contract window by moving 'left' pointer until constraint is satisfied again.

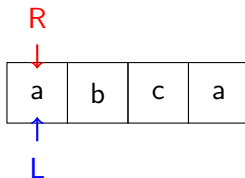
# **Problem 1: Longest Substring Without Repeating Characters**

# Longest Substring Without Repeating Characters

**Problem:** Find length of the longest substring with unique characters.

- **Expand:** Move right pointer.

## Step-by-Step Window



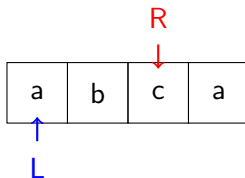
*Expanding window...*

# Longest Substring Without Repeating Characters

**Problem:** Find length of the longest substring with unique characters.

- ▶ **Expand:** Move right pointer.
- ▶ **Detect:** If  $s[\text{right}]$  is in Map...

## Step-by-Step Window



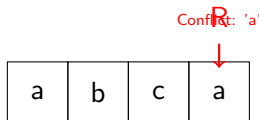
*Expanding window...*

# Longest Substring Without Repeating Characters

**Problem:** Find length of the longest substring with unique characters.

- ▶ **Expand:** Move right pointer.
- ▶ **Detect:** If `s[right]` is in Map...
- ▶ **Jump:** Move left to `Map[char] + 1`.

## Step-by-Step Window



*Duplicate 'a' found! Jump L.*

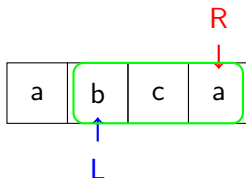


# Longest Substring Without Repeating Characters

**Problem:** Find length of the longest substring with unique characters.

- ▶ **Expand:** Move right pointer.
- ▶ **Detect:** If `s[right]` is in Map...
- ▶ **Jump:** Move left to `Map[char] + 1`.
- ▶ **Update:** Update Max Length.

## Step-by-Step Window



*New Window: "bca"*

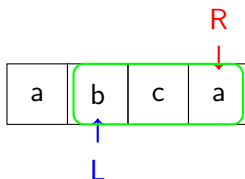
# Longest Substring Without Repeating Characters

**Problem:** Find length of the longest substring with unique characters.

- ▶ **Expand:** Move right pointer.
- ▶ **Detect:** If `s[right]` is in Map...
- ▶ **Jump:** Move left to `Map[char] + 1`.
- ▶ **Update:** Update Max Length.

```
for (right in s.indices)
{
    if (map.containsKey(s
        [right])) {
        left = maxOf(left
            , map[s[right]
                ]!! + 1)
    }
}
```

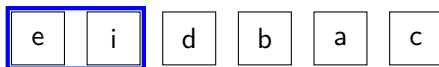
## Step-by-Step Window



## **Problem 2: Permutation in String**

## Permutation in String: Fixed Window

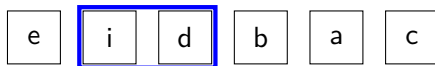
**Problem:** Does  $s_2$  contain a permutation of  $s_1$ ?



Window 1

## Permutation in String: Fixed Window

**Problem:** Does  $s_2$  contain a permutation of  $s_1$ ?



Window 2

## Permutation in String: Fixed Window

**Problem:** Does  $s_2$  contain a permutation of  $s_1$ ?



Permutation Found!

# Permutation in String: Fixed Window

**Problem:** Does  $s_2$  contain a permutation of  $s_1$ ?



## HashMap Slide Logic

```
// 1. Add right character
s2Map[s2[right]] = s2Map.getOrDefault(s2[right], 0) + 1
// 2. Remove left character
if (s2Map[s2[left]] == 1) s2Map.remove(s2[left])
else s2Map[s2[left]] = s2Map[s2[left]]!! - 1
// 3. Compare Maps
if (s1Map == s2Map) return true
```

## **Problem 3: Fruit Into Baskets**



## **Problem 4: Minimum Window Substring**

## Section 5 — Monotonic Stack

# What is a Monotonic Stack?

- ▶ **The Core Idea:** A stack that maintains elements in a strictly increasing or decreasing order.
- ▶ **How it works:** When a new element is processed, we pop all elements from the stack that violate the monotonic property before pushing the new element.
- ▶ **Common Triggers:**
  - ▶ Finding the "next greater" or "previous smaller" element in an array.
  - ▶ Computing areas bounded by heights (e.g. histograms, trapping water).
  - ▶ Resolving queries in  $O(N)$  total time where each index is pushed/popped at most once.

# Monotonic Stack: Types

- ▶ **Monotonic Increasing Stack:** Stack elements are sorted in ascending order from bottom to top (e.g. '[1, 3, 5, 8]'). Useful to find the first smaller element to the left/right.
- ▶ **Monotonic Decreasing Stack:** Stack elements are sorted in descending order from bottom to top (e.g. '[9, 7, 4, 2]'). Useful to find the first larger element to the left/right.

## **Problem 1: Daily Temperatures**

## **Problem 2: Next Greater Element I**

## **Problem 3: Trapping Rain Water**

## **Problem 4: Largest Rectangle in Histogram**



## **Section 6 — Prefix Sum & Cumulative Patterns**

# What is the Prefix Sum Pattern?

- ▶ **The Core Idea:** Precompute cumulative sums of an array: 'prefix[i] = nums[0] + ... + nums[i]'.
- ▶ **Why it works:** Allows you to calculate the sum of any subarray 'nums[i...j]' in  $O(1)$  time:

$$\text{Sum}(i, j) = \text{prefix}[j] - \text{prefix}[i - 1]$$

- ▶ **Common Triggers:**
  - ▶ Range sum query problems with static arrays.
  - ▶ Finding subarrays that sum to a target value  $K$ .
  - ▶ Subarrays with properties related to divisibility or modulo arithmetic.

## Combining Prefix Sum with HashMap

To find subarrays summing to  $K$  in  $O(N)$  time:

- ▶ Maintain running cumulative sum 'currentSum'.
- ▶ At each step, we look for a previous prefix sum that equals 'currentSum -  $K$ '.
- ▶ Use a HashMap to store frequencies of prefix sums seen so far.
- ▶ **Key Modulo Trick:** For divisibility by  $K$ , store 'currentSum

## **Problem 1: Subarray Sum Equals K**

## **Problem 2: Subarray Sums Divisible by K**

## Section 7 — Cyclic Sort

# What is the Cyclic Sort Pattern?

- ▶ **The Core Idea:** When an array contains numbers in a range from 1 to  $n$  (or 0 to  $n$ ), we can sort it in  $O(n)$  time and  $O(1)$  auxiliary space.
- ▶ **How it works:** Iterate through the array. If the current number 'nums[i]' is not at its correct index (i.e. 'nums[i]  $\neq$  i + 1'), swap it with the number at its correct index 'nums[nums[i] - 1]'. Repeat this until the correct number is at index 'i', then move to the next index.
- ▶ **Common Triggers:**
  - ▶ Problems dealing with numbers in a continuous range.
  - ▶ Finding duplicate or missing numbers in an array.
  - ▶ Finding the first/smallest missing positive number.

# Cyclic Sort Mechanics

- ▶ **Index Rule:** A value 'val' belongs at index 'val - 1' (or index 'val' if range is 0 to  $n$ ).
- ▶ **Loop condition:** We only swap when the target index has a different value:

$$\text{nums}[i] \neq \text{nums}[\text{nums}[i] - 1]$$

- ▶ **Complexity analysis:** Although we have a nested loop or repeat swaps at the same index, each swap puts at least one number in its correct position. Thus, at most  $N - 1$  swaps are performed, leading to  $O(N)$  time.



## **Problem 1: Find All Numbers Disappeared in an Array**

## **Problem 2: First Missing Positive**

## **Problem 3: Find All Duplicates in an Array**

## Section 8 — Merge Intervals

# What is the Merge Intervals Pattern?

- ▶ **The Core Idea:** Process overlapping intervals by sorting them and merging adjacent ranges.
- ▶ **Why it works:** Once sorted by start time, overlapping intervals will be adjacent. We can merge them in a single linear pass.
- ▶ **Common Triggers:**
  - ▶ Finding overlapping ranges, free slots, or busy schedules.
  - ▶ Inserting new intervals into already sorted ranges.
  - ▶ Finding intersections of multiple lists of intervals.

## Merging Logic

Given two sorted intervals  $A$  and  $B$ , they overlap if:

$$B.start \leq A.end$$

If they overlap, we merge them into a single interval  $C$  where:

$$C.start = A.start$$

$$C.end = \max(A.end, B.end)$$

Otherwise, they do not overlap, and we add  $A$  to our output list and set  $B$  as the new active interval.

## **Problem 1: Merge Intervals**

## **Problem 2: Insert Interval**



## **Problem 3: Employee Free Time**

## **Section 9 — Modified Binary Search**

# What is Modified Binary Search?

- ▶ **The Core Idea:** An extension of standard binary search that applies to non-standard sorted structures or search spaces.
- ▶ **Why it works:** By dividing the search space in half in  $O(1)$  time, we achieve a logarithmic  $O(\log N)$  runtime.
- ▶ **Common Triggers:**
  - ▶ Searching in a rotated sorted array.
  - ▶ Finding boundaries, peaks, or transition points.
  - ▶ **Binary Search on Answers:** When checking if a candidate answer is valid takes  $O(N)$  time, and the answer is monotonic, we search the answer range.

# Binary Search on Answer Space

Instead of searching indices in an array, we search value ranges (from 'minPossibleAnswer' to 'maxPossibleAnswer'):

- ▶ Find the midpoint 'mid' of the current range.
- ▶ Run a validation function 'isValid(mid)' to check if 'mid' is a possible solution (usually in  $O(N)$  time).
- ▶ Based on the result:
  - ▶ If valid: Try to find a better solution in the lower half (if minimizing).
  - ▶ If invalid: Discard the lower half and search the upper half.

## **Problem 1: Search in Rotated Sorted Array**

## **Problem 2: Find Minimum in Rotated Sorted Array**

## **Problem 3: Capacity To Ship Packages Within D Days**

## Problem 4: Pow(x, n)



## **Section 10 — Fast & Slow Pointers**

# What is the Fast & Slow Pointers Pattern?

- ▶ **The Core Idea:** Use two pointers traversing a linear data structure (linked list/array) at different speeds.
- ▶ **Why it works:** If there is a cycle, the fast pointer (moving 2 steps at a time) will eventually meet the slow pointer (moving 1 step at a time) in  $O(N)$  time and  $O(1)$  space.
- ▶ **Common Triggers:**
  - ▶ Cycle detection in a linked list.
  - ▶ Finding the starting node of a cycle.
  - ▶ Finding the middle node of a list (when the fast pointer reaches the end, the slow pointer is in the middle).
  - ▶ Identifying duplicates in structures where values can be treated as indices.

# Cycle Detection Mechanics

- ▶ **Initialize:** 'slow = head', 'fast = head'.
- ▶ **Advance:**
  - ▶ 'slow = slow.next'
  - ▶ 'fast = fast.next.next'
- ▶ **Check:** If 'slow == fast', a cycle is detected! If 'fast' or 'fast.next' is null, there is no cycle.
- ▶ **Cycle Start (Floyd's algorithm):** Once met, reset 'slow' to 'head'. Move both pointers 1 step at a time. The point where they meet again is the start of the cycle.

## **Problem 1: Linked List Cycle**

## **Problem 2: Middle of the Linked List**

## **Problem 3: Find the Duplicate Number**

## **Section 11 — Tree Depth First Search (DFS)**

# Tree Depth First Search (DFS)

- ▶ **The Core Idea:** Explore as deep as possible along each branch of a tree before backtracking.
- ▶ **Implementation:** Typically implemented using recursion (which uses the call stack implicitly) or an explicit Stack.
- ▶ **Common Triggers:**
  - ▶ Finding paths that sum to a target value.
  - ▶ Computing properties of nodes recursively (height, balance).
  - ▶ Validating BST properties.
  - ▶ Exploring combinations/subtrees.



# DFS Traversal Types

Depending on when the current node is processed relative to its children:

- ▶ **Pre-order (Root, Left, Right):** Processing parent before children. Useful for copying trees or serialization.
- ▶ **In-order (Left, Root, Right):** Processing left child, then parent, then right child. Yields sorted keys in a Binary Search Tree (BST).
- ▶ **Post-order (Left, Right, Root):** Processing children before parent. Useful for bottom-up computation (e.g., node height, subproblem accumulation).

## **Problem 1: Path Sum**

## Problem 2: Path Sum III

## **Problem 3: Validate Binary Search Tree**

## **Problem 4: Lowest Common Ancestor of a Binary Tree**

## **Problem 5: Binary Tree Maximum Path Sum**

## **Section 12 — Tree Breadth First Search (BFS)**

# Tree Breadth First Search (BFS)

- ▶ **The Core Idea:** Traverse a tree level-by-level, processing all nodes at depth  $d$  before any node at depth  $d + 1$ .
- ▶ **Implementation:** Typically implemented iteratively using a Queue (FIFO).
- ▶ **Common Triggers:**
  - ▶ Level order traversals or printing.
  - ▶ Finding the minimum depth of a tree.
  - ▶ Tracking "level" boundaries (e.g. connecting next pointers).
  - ▶ Finding the shortest path in unweighted graphs or state transitions.



## Level Size Strategy

To process nodes level by level rather than as a continuous queue stream:

- ▶ At the start of each level iteration, count the queue size: `'val levelSize = queue.size'`.
- ▶ Run a nested loop exactly `'levelSize'` times.
- ▶ Inside the loop:
  - ▶ Pop a node from the queue.
  - ▶ Enqueue its children (left, then right).
- ▶ Any logic scoped to a level (e.g., calculating level average or reversing order) can be done cleanly.

## **Problem 1: Binary Tree Level Order Traversal**

## **Problem 2: Binary Tree Zigzag Level Order Traversal**

## **Problem 3: Populating Next Right Pointers in Each Node**

## **Problem 4: Word Ladder**

## **Section 13 — Matrix Traversal (The "Islands" Pattern)**

# What is the Matrix Traversal Pattern?

- ▶ **The Core Idea:** Treat a 2D grid (matrix) as an implicit graph where each cell  $(r, c)$  is a node, and its adjacent neighbors (up, down, left, right) are connected by edges.
- ▶ **Traversal:** Use Depth First Search (DFS) or Breadth First Search (BFS) to explore connected regions.
- ▶ **Common Triggers:**
  - ▶ Counting distinct blocks (e.g. islands, provinces).
  - ▶ Simulating cell expansion over time (e.g. rotting oranges, fire spreading).
  - ▶ Finding paths or reachable regions from boundaries.

# Matrix Traversal Mechanics

- ▶ **Boundary Check:** Always ensure row 'r' and column 'c' are valid:

$$0 \leq r < R \quad \text{and} \quad 0 \leq c < C$$

- ▶ **Directions Array:** Use a utility array to iterate over neighbors cleanly:

$$\text{dirs} = [(-1, 0), (1, 0), (0, -1), (0, 1)]$$

- ▶ **Visited Tracking:** Mark grid cells as visited (e.g., set "1" to "0" in-place if allowed, or use a 'visited' boolean matrix) to prevent infinite loops.



## **Problem 1: Number of Islands**

## **Problem 2: Max Area of Island**

## **Problem 3: Rotting Oranges**

## **Problem 4: Pacific Atlantic Water Flow**

## Section 14 — Union-Find (Disjoint Set)

# What is the Union-Find Pattern?

- ▶ **The Core Idea:** A data structure that tracks partition of a set into disjoint subsets.
- ▶ **Operations:**
  - ▶ **Find:** Identify which subset an element belongs to (returns root parent).
  - ▶ **Union:** Join two disjoint subsets into a single subset.
- ▶ **Common Triggers:**
  - ▶ Dynamic connectivity (are nodes  $X$  and  $Y$  in the same group?).
  - ▶ Detecting cycles in undirected graphs.
  - ▶ Merging accounts, groups, or equivalence sets.

# Optimizations: Path Compression & Union by Rank

Without optimization, a tree can become a line ( $O(N)$  lookup).

- ▶ **Path Compression:** During 'find(x)', make every node in the search path point directly to the root root. This flattens the tree structure:

$$\text{parent}[x] = \text{find}(\text{parent}[x])$$

- ▶ **Union by Rank/Size:** Always attach the smaller tree under the root of the larger tree.
- ▶ **Complexity:** Combined, they achieve nearly constant time operations:  $O(\alpha(N))$  where  $\alpha$  is the Inverse Ackermann function.

## **Problem 1: Number of Provinces**



## **Problem 2: Redundant Connection**

## **Problem 3: Accounts Merge**

## **Section 15 — Top 'K' Elements (Heap / Priority Queue)**

# What is the Top 'K' Elements Pattern?

- ▶ **The Core Idea:** Find the  $K$  largest, smallest, or most frequent elements in a large collection.
- ▶ **Why it works:** Instead of sorting the entire array in  $O(N \log N)$  time, we can maintain a Heap/Priority Queue of size  $K$ .
- ▶ **Heaps selection rule:**
  - ▶ To find the  **$K$  largest** elements: Use a **Min-Heap**. When size exceeds  $K$ , pop the smallest element. The heap will retain the  $K$  largest values.
  - ▶ To find the  **$K$  smallest** elements: Use a **Max-Heap**. When size exceeds  $K$ , pop the largest.
- ▶ **Complexity:**  $O(N \log K)$  time and  $O(K)$  extra space.

## **Problem 1: Top K Frequent Elements**

## **Problem 2: K Closest Points to Origin**

## **Problem 3: Task Scheduler**

## **Problem 4: Reorganize String**



## **Section 16 — Dynamic Programming**

# What is Dynamic Programming (DP)?

- ▶ **The Core Idea:** Solve complex problems by breaking them down into simpler, overlapping subproblems and caching their results to avoid redundant calculations.
- ▶ **Key Properties:**
  - ▶ **Overlapping Subproblems:** Solving the same subproblems repeatedly.
  - ▶ **Optimal Substructure:** The optimal solution of the problem can be constructed from optimal solutions of its subproblems.
- ▶ **Approaches:**
  - ▶ **Top-Down (Memoization):** Recursive, start from target, solve subproblems and save values in a cache.
  - ▶ **Bottom-Up (Tabulation):** Iterative, start from base cases, fill a table (array/matrix) up to target.

# Dynamic Programming Sub-patterns

Most interview DP problems fall into standard subclasses:

- ▶ **Fibonacci Numbers:**  $dp[i] = dp[i-1] + dp[i-2]$  (Climbing Stairs).
- ▶ **0/1 Knapsack:** Items can be chosen at most once. Decide to include or exclude an item:  $dp[i][w] = \max(\text{exclude}, \text{include})$ .
- ▶ **Unbounded Knapsack / Coin Change:** Items can be chosen multiple times.
- ▶ **Longest Common Subsequence (LCS) / Edit Distance:** String matching.
- ▶ **Longest Increasing Subsequence (LIS):** Decisions based on previous elements.

## **Problem 1: Climbing Stairs**

## **Concept: 0/1 Knapsack**

## **Problem 3: Coin Change**

## **Problem 4: Coin Change 2**

## **Problem 5: Longest Increasing Subsequence**



## **Problem 6: Word Break**

## Section 17 — Backtracking & DFS

# What is the Backtracking Pattern?

- ▶ **The Core Idea:** A systematic search algorithm that builds candidate solutions incrementally, and abandons a candidate ("backtracks") as soon as it determines the candidate cannot lead to a valid solution.
- ▶ **How it works (The Three Steps):**
  1. **Choose:** Add an element to the current path.
  2. **Explore:** Recursively call the search function with the new state.
  3. **Unchoose (Backtrack):** Remove the element from the current path to return to the previous state.
- ▶ **Common Triggers:**
  - ▶ Generating all permutations, combinations, or subsets.
  - ▶ Solving constraint satisfaction puzzles (e.g. N-Queens, Sudoku).
  - ▶ Exploring word combinations on a grid.

## **Problem 1: Generate Parentheses**

## **Problem 2: Combination Sum**

## **Problem 3: Palindrome Partitioning**

## **Problem 4: Word Search**

## Problem 5: N-Queens



## **Section 18 — Greedy Algorithms**

# What are Greedy Algorithms?

- ▶ **The Core Idea:** An algorithmic paradigm that builds up a solution piece by piece, always choosing the next piece that offers the most obvious and immediate local benefit ("greedy" choice).
- ▶ **Why it works:** For certain problems, making locally optimal decisions leads to a globally optimal solution.
- ▶ **Tradeoffs:**
  - ▶ Highly efficient (usually  $O(N)$  or  $O(N \log N)$ ).
  - ▶ Difficult to mathematically prove correctness.
  - ▶ If greedy fails, you must fall back to Dynamic Programming (which checks all combinations).
- ▶ **Common Triggers:**
  - ▶ Scheduling, interval selection, minimizations.
  - ▶ Gas stations, container loading, coin distributions (canonical cases).

## **Problem 1: Gas Station**

## **Problem 2: Candy**

## **Problem 3: Queue Reconstruction by Height**

## **Section 19 — Data Streams & Online Algorithms**

# Data Streams & Online Algorithms

- ▶ **The Core Idea:** Process data that arrives continuously over time, rather than having the entire dataset available in memory upfront.
- ▶ **Constraint:** Algorithms must run in real-time with limited space ( $O(1)$  or  $O(K)$  where  $K$  is the query window size) and time complexity per element.
- ▶ **Key Techniques:**
  - ▶ **Two Heaps:** Balancing a Max-Heap and Min-Heap to find median in  $O(1)$  time.
  - ▶ **Reservoir Sampling:** Selecting a random sample from a stream of unknown size.
  - ▶ **Double Linked List + HashMap:** Designing fast cache policies (like LRU/LFU cache) with  $O(1)$  insertion, lookup, and deletion.

## **Problem 1: Find Median from Data Stream**



## **Problem 2: Random Pick Index**

## **Problem 3: Moving Average from Data Stream**

## Problem 4: LRU Cache

## **Section 20 — Advanced Patterns**

# Advanced Algorithmic Patterns

For Senior and Staff level roles, interviewers often present advanced structures:

- ▶ **Segment Tree:** A binary tree used for storing intervals or segments. It allows querying a range (sum, min, max) and modifying elements in  $O(\log N)$  time.
- ▶ **Topological Sort:** Ordering vertices in a directed acyclic graph (DAG) such that for every directed edge  $uv$ , vertex  $u$  comes before  $v$ . (Kahn's BFS or DFS).
- ▶ **Sweep Line:** A geometry paradigm that solves range/interval problems by sorting boundary events (start/end) and sweeping a imaginary line across them.
- ▶ **Trie (Prefix Tree):** A search tree used to store associative keys (typically strings) efficiently. Excellent for autocompletion, spelling checkers, prefix matching.

## **Segment Tree: Range Sum Query - Mutable**

# **Topological Sort: Course Schedule & Course Schedule II**

## **Sweep Line: The Skyline Problem**



## **Tries: Implement Trie & Replace Words**

## **Section 21 — Miscellaneous Techniques**

# Miscellaneous Interview Techniques

Some interview questions don't fit into single patterns, but require a combination of general skills:

- ▶ **In-place Grid Simulation:** Modifying a matrix without using extra memory by using rows/columns headers as state markers (e.g. Set Matrix Zeroes).
- ▶ **Math & Coordinate Transformations:** Mirroring, swapping, or transposing grids (e.g. Rotate Image).
- ▶ **Stack Evaluation:** Parsing arithmetic strings or expressions sequentially (e.g. Evaluate Reverse Polish Notation).
- ▶ **Bitmasking or Row/Column hashing:** Validating board boundaries quickly (e.g. Valid Sudoku).

## **Problem 1: Set Matrix Zeroes**

## **Problem 2: Valid Sudoku**

## **Problem 3: Rotate Image**

## **Problem 4: Evaluate Reverse Polish Notation**

## **Section 22 — Summary & The Modern Interview Landscape**



# Systematic Problem-Solving Template

How to tackle any unknown problem in an interview:

1. **Clarify:** Ask questions about constraints ( $n$ , value limits, sorted status, memory limits).
2. **Example:** Dry run a custom, non-trivial test case.
3. **Brute Force:** State the obvious solution ( $O(N^2)$  or  $O(2^N)$ ), explaining its bottleneck.
4. **Optimize:** Apply pattern matching (e.g. "Do I need a window?", "Can a map help?").
5. **Code:** Write clean, modular code, stating edge cases.
6. **Dry Run:** Trace the code line-by-line with the example.

# The Engineering Manager's Lens

Interviews are a signal of how you will perform as a team member:

- ▶ **\*\*Tradeoffs:\*\*** Be vocal about time vs. space complexity tradeoffs.
- ▶ **\*\*Production-Ready Code:\*\*** Favor clean modular helper methods, input validation, and proper typing over dense one-liners.
- ▶ **\*\*Collaboration:\*\*** Treat the interviewer as a teammate. Don't go silent for minutes; explain your thought flow continuously.

# Redefining the SDLC in the AI Era

How AI coding agents (like Gemini, Copilot) change the baseline expectation:

- ▶ **\*\*Syntax is Cheap:\*\*** Being a human compiler or memorizing syntax is no longer the primary value.
- ▶ **\*\*Logic & System Design is King:\*\*** Focus on high-level reasoning, system architecture, identifying constraints, and verifying correctness.
- ▶ **\*\*Coding speed vs. System security:\*\*** Writing code quickly is easy; ensuring it is robust, secure, and fits into the existing framework is the real challenge.

# Recap & Roadmap Beyond the Course

- ▶ You have studied 20+ crucial algorithmic patterns!
- ▶ **\*\*Next Steps:\*\***
  - ▶ Practice spaced repetition (solve a problem, review it after 3 days, then 7 days).
  - ▶ Conduct mock whiteboard interviews with peers.
  - ▶ Build intuition over syntax.

**Congratulations on completing the course!**